



Fundamentos de Inteligencia Artificial

Complejidad de algoritmos y
calculabilidad de problemas

José A. Montenegro

13 de marzo de 2026

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA DE TELECOMUNICACIÓN



Contenido

Formalización del concepto de algoritmo

Lenguaje While

- Concepto informal

- Concepto formal

- Semántica

- Computabilidad

- Ejercicios

- Codificación de programas WHILE

- Problema de la Parada

- Función castor afanoso

Complejidad

- Complejidad Temporal

Calculabilidad con Python



Formalización del concepto de algoritmo





Formalización del concepto de algoritmo



En el siglo VIII, el califa de Bagdad Hârûn al-Rasîd fundó “La casa de la Sabiduría”, impulsada luego por su hijo el califa al-Ma’mûn (786-833), como un lugar para el desarrollo y protección del saber y los sabios.

Abu Ja’far Mohammed ibn Musa al-Khowârizmî (780-850) fue miembro de la casa de la sabiduría, de su nombre procede la palabra **algoritmo** cuya forma original era **algorismo**.



En abril de 1935, *Alonzo Church* presentó una comunicación donde formalizaba las funciones recursivas en términos del λ -**cálculo** y enunció la equivalencia entre su clase de funciones y las calculables por procedimientos efectivos (hoy conocida como tesis de Church-Turing).



Formalización del concepto de algoritmo



En septiembre de 1935 *Kleene* presentó en una comunicación donde definió **las funciones recursivas primitivas** y las **funciones recursivas generales** y demostraba el teorema de la Forma Normal donde demostró que cada programa admite uno equivalente usando bucles FOR y un sólo bucle WHILE.



En 1936 *Alan Turing* (1912-1954) da una nueva formalización definiendo lo que hoy se conoce como máquina de Turing. Turing entre otros elementos trató **el problema de la parada**, definió el concepto de “**paso de cómputo**”, el cual es fundamental para el desarrollo de la teoría de la complejidad.

Posteriormente a los **modelos de Church, Kleene y Turing**, han aparecido otros con diferentes orientaciones. Uno de ellos es el **lenguaje WHILE**, que tiene sentencias de asignación y una sola de control que le da nombre al lenguaje, el bucle WHILE.



Formalización del concepto de algoritmo

- Según la **tesis de Church-Turing**, las **funciones computables** son exactamente las funciones que pueden calcularse utilizando un dispositivo de cálculo mecánico dadas cantidades ilimitadas de tiempo y espacio de almacenamiento.
- Equivalentemente, esta tesis afirma que una **función es computable** si y sólo si tiene un **algoritmo**.
 - Así, podemos decir que un **algoritmo** es una secuencia finita de instrucciones precisas (no ambiguas) que, para un cierto tipo de entrada, o bien nos devuelve (computa en un tiempo finito) **una salida** o bien **no termina nunca**.
 - Si un algoritmo siempre, para su tipo de entrada, nos devuelve en un tiempo finito una salida, entonces decimos que es un **algoritmo conclusivo**.
 - Diremos que una **función es computable** si existe un algoritmo que para cualquier argumento de la función como entrada nos devuelve el valor de la función como salida.
 - Si la función para cierto argumento no toma valor (es decir, es una **función parcial**) entonces el **algoritmo** para ese argumento como entrada **no termina nunca**.
 - De aquí se deduce que si una **función** computable es **total** entonces el algoritmo que la computa es un **algoritmo conclusivo**.



Formalización del concepto de algoritmo

- Sea A un conjunto de cadenas sobre un alfabeto y B un subconjunto de A .
- Diremos que B es un **conjunto decidable** si existe un algoritmo conclusivo que dado un elemento de A como entrada nos dice siempre (en un tiempo finito al ser el algoritmo conclusivo) si dicho elemento pertenece o no al conjunto B .
- Diremos que B es un **conjunto enumerable** si existe un algoritmo que dado un elemento de A como entrada nos devuelve una respuesta afirmativa si y sólo si dicho elemento pertenece al conjunto B .
- La diferencia entre **decidable** y **enumerable** es que en el primer caso siempre tenemos respuesta del algoritmo, mientras que en el segundo sólo podemos asegurar que el algoritmo terminará si el elemento en cuestión pertenece al conjunto.
- De aquí se deduce que todo conjunto decidable es enumerable pero no a la inversa.



Lenguaje While





Concepto informal



Concepto informal

Para formalizar el concepto de función computable (WHILE-computable) definimos previamente un modelo de cómputo el lenguaje WHILE.

- El lenguaje WHILE se puede entender como un lenguaje de programación muy simplificado.
- Sólo trabaja con naturales



Concepto informal

- **Cuatro tipos de sentencias** de asignación,
 - $X_i := 0$ asignar a una variable la constante cero
 - $X_i := X_j$ asignar a una variable el valor de otra variable,
 - $X_i := X_j + 1$ asignar a una variable el valor de otra variable incrementada en una unidad,
 - $X_i := X_j - 1$ y asignar a una variable el valor de otra variable decrementada en una unidad.
- Un **tipo de sentencia de control**. condición valor de una variable (denominada de control) es distinto de cero.
 - while $X_i \neq 0$ x do (es la llamada cabecera del bucle)
 - código (es el llamado cuerpo del bucle)
 - od (es la llamada cola del bucle)



Concepto informal

- En un programa WHILE los identificadores de variables tienen la forma: $X_1, X_2, X_3, \dots, X_i, \dots$
- Un código es una **secuencia finita no vacía de sentencias** separadas por el símbolo “;”.
- Un programa WHILE es una terna (n, p, s) donde
 - n y p son naturales con $p \geq n$ y $p > 0$;
 - s es un código según la sintaxis anteriormente descrita.



Concepto informal

Un programa WHILE (n, p, s) esto significa que el programa tiene:

- p **variables de uso en total** $(X_1, X_2, X_3, \dots, X_p)$ que toman valores naturales, de ellas:
- n son **variables de entrada** $(X_1, X_2, X_3, \dots, X_n)$, denominándose a
- $p-n$ las restantes **variables auxiliares**. Se inicializan implícitamente a cero al inicio de la ejecución del programa.
- No hay instrucciones de entrada ni de salida.
- La salida del programa es el valor de la variable X_1 cuando el programa termina



Concepto informal

El funcionamiento de un programa WHILE consiste, dado que es un modelo basado en un lenguaje estructurado, en ejecutar secuencialmente las sentencias que componen su código, empezando por la primera del mismo.

- Se ejecuta una sentencia cuando la anterior ha sido terminada.
- La sentencia de **decremento** ($X_i := X_j - 1$) asigna el valor cero a la variable resultante en el caso de que la variable decrementada valiera cero antes de la sentencia.
- Respecto a la sentencia de control (while), ésta primero comprueba si la variable de control que aparece en su cabecera es distinta de cero; si es así ejecutará el código que compone el cuerpo del bucle, tras lo cual volverá a comprobar la variable. Esto se repetirá mientras dicha variable sea distinta de cero cada vez que sea comprobada.



Concepto informal

- Si en una de las comprobaciones vale cero la sentencia while termina.
- Podemos por tanto asociar una única función de N^n en N a cada programa WHILE, **función calculada por un programa WHILE.**
- Función puede ser **parcial**, ya que puede haber entradas para las cuales el programa no termina (diverge).



Ejemplo

Sea $Q = (1, 2, s)$ con s :

$X_2 := X_1$;

while $X_2 \neq 0$ do

$X_1 := X_1 + 1$;

$X_2 := X_2 - 1$

od



Concepto formal: Sintáxis



Concepto formal:Sintáxis

Tenemos dos alfabetos:

- $\Sigma_d = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}$. El alfabeto Σ_d está formado por los dígitos en subíndice.
- $\Sigma_1 = \{ X, :=, +, -, 1, \text{while}, \neq, 0, \text{do}, \text{od}, ; \}$ “while” es un solo símbolo perteneciente al alfabeto Σ_1 , que tiene por tanto once símbolos.



Concepto formal: Sintaxis

Tenemos dos gramáticas:

Sea $G_1 = (N_1, T_1, P_1, \langle \text{numero} \rangle)$ una gramática con:

$N_1 = \{ \langle \text{numero} \rangle, \langle \text{p_dígito} \rangle, \langle \text{r_numero} \rangle, \langle \text{dígito} \rangle \}$

$T_1 = \Sigma_d$

$P_1 = \{ \langle \text{numero} \rangle \rightarrow \langle \text{p_dígito} \rangle$

$| \langle \text{p_dígito} \rangle \langle \text{r_numero} \rangle,$

$\langle \text{p_dígito} \rangle \rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9,$

$\langle \text{r_numero} \rangle \rightarrow \langle \text{dígito} \rangle$

$| \langle \text{dígito} \rangle \langle \text{r_numero} \rangle,$

$\langle \text{dígito} \rangle \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \}$



Concepto formal: Sintaxis

Sea $G_2 = (N_2, T_2, P_2, \langle \text{código} \rangle)$ una gramática con:

$$N_2 = \{ \langle \text{código} \rangle, \langle \text{sentencia} \rangle, \langle \text{asignación} \rangle, \langle \text{bucle} \rangle, \langle \text{identificador} \rangle \} \cup N_1$$
$$T_2 = \Sigma_d \cup \Sigma_1$$
$$P_2 = \{ \langle \text{código} \rangle \rightarrow \langle \text{sentencia} \rangle$$
$$| \langle \text{código} \rangle ; \langle \text{sentencia} \rangle,$$
$$\langle \text{sentencia} \rangle \rightarrow \langle \text{asignación} \rangle$$
$$| \langle \text{bucle} \rangle,$$
$$\langle \text{asignación} \rangle \rightarrow \langle \text{identificador} \rangle := 0$$
$$| \langle \text{identificador} \rangle := \langle \text{identificador} \rangle$$
$$| \langle \text{identificador} \rangle := \langle \text{identificador} \rangle + 1$$
$$| \langle \text{identificador} \rangle := \langle \text{identificador} \rangle - 1,$$
$$\langle \text{bucle} \rangle \rightarrow \text{while } \langle \text{identificador} \rangle \neq 0 \text{ do } \langle \text{código} \rangle \text{ od},$$
$$\langle \text{identificador} \rangle \rightarrow X \langle \text{número} \rangle \} \cup P_1$$



Concepto formal: Sintáxis

Definición 1: Código While, Conjunto de códigos While (Cód_While)

Sea $s \in T_2^+$. Diremos que s es un código While sii $s \in L(G_2)$.

El conjunto de códigos While, notado Cód_While, se define como: $\text{Cód_While} = L(G_2)$.

Nota: El conjunto Cód_While está formado por todos los códigos en lenguaje WHILE sintácticamente correctos.

Definición 2: Programa WHILE, Conjunto de programas WHILE (WHILE)

El conjunto de programas WHILE, notado WHILE, se define como:

$$\text{WHILE} = \{ (n, p, s) \in \mathbb{N} \times \mathbb{N} \times \text{Cód_While} \mid p \geq n \wedge p > 0 \wedge (X_m \succ s \text{ con } m \in \Sigma_d^+ \Rightarrow m \leq p) \}$$

Diremos que Q es un programa WHILE sii $Q \in \text{WHILE}$.

Nota: La expresión " $X_m \succ s$ " significa que la cadena " X_m " es subcadena de la cadena " s ".



Concepto formal: Semántica



Concepto formal: Semántica

Definición 3: Tamaño de un código While (tam)

Sea $s \in \text{Cód_While}$, definimos el tamaño de s , notado $\text{tam}(s)$, como:

- Si en G_2 se cumple que $\langle \text{asignación} \rangle \Rightarrow^* s$, entonces: $\text{tam}(s) = 1$
- Si $s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od}$, con $i \in \Sigma_d^+$ y $s_1 \in \text{Cód_While}$, entonces: $\text{tam}(s) = \text{tam}(s_1) + 2$
- Si $s = s_1 ; s_2$, con $s_1, s_2 \in \text{Cód_While}$, entonces: $\text{tam}(s) = \text{tam}(s_1) + \text{tam}(s_2)$

Definición 4: Tamaño de un programa WHILE (tam)

Sea $Q \in \text{WHILE}$, con $Q = (n, p, s)$. Definimos el tamaño de Q , notado $\text{tam}(Q)$, como: $\text{tam}(Q) = \text{tam}(s)$



Volvemos con el Ejemplo

Sea $Q = (1, 2, s)$ con s :

```
 $X_2 := X_1;$ 
```

```
while  $X_2 \neq 0$  do
```

```
     $X_1 := X_1 + 1;$ 
```

```
     $X_2 := X_2 - 1$ 
```

```
od
```

$$\text{WHILE} = \{ (n, p, s) \in \mathbb{N} \times \mathbb{N} \times \text{Cód_While} \mid p \geq n \wedge p > 0 \wedge (X_m \succ s \text{ con } m \in \Sigma_d^+ \Rightarrow m \leq p) \}$$

$Q \in \text{WHILE}$, es decir, es un programa WHILE, ya que se cumple que:

$$Q \in \mathbb{N} \times \mathbb{N} \times \text{Cód_While} \wedge 2 \geq 1 \wedge p \geq n \wedge 2 > 0 \wedge p > 0 \wedge (X_m \succ s \text{ con } m \in \Sigma_d^+ \Rightarrow m \leq 2)$$

El hecho de que la tercera componente de Q , es decir, la cadena s , pertenezca a Cód_While

- significa que s es un código en lenguaje WHILE sintácticamente correcto,
- cumpliéndose que $\text{tam}(Q) = \text{tam}(s) = 5$



Función Línea ...

Definición 5: Función línea...

Definimos la función **línea** : $\text{Cód_While} \times N \rightarrow T_2^*$ como:

1. Si en G_2 se cumple que $\langle \text{asignación} \rangle \Rightarrow^* s$, entonces:

$$\text{linea}(s, n) = \begin{cases} s & \text{si } n=1 \\ \epsilon & \text{en otro caso} \end{cases}$$

2. Si $s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od}$, con $i \in \Sigma_d^+$ y $s_1 \in \text{Cód_While}$, entonces:

$$\text{linea}(s, n) = \begin{cases} \text{while } X_i \neq 0 \text{ do} & \text{si } n=1 \\ \text{od} & \text{si } n=\text{tam}(s) \\ \text{linea}(s_1, n-1) & \text{si } 1 < n < \text{tam}(s) \\ \epsilon & \text{en otro caso} \end{cases}$$



... Función Línea

Definición 5: .. Función línea

3. Si $s = s_1 ; s_2$, con $s_1, s_2 \in \text{Cód_While}$, entonces:

$$\text{linea}(s, n) = \begin{cases} \text{linea}(s_1, n) & \text{si } 0 < n \leq \text{tam}(s_1) \\ \text{linea}(s_2, n - \text{tam}(s_1)) & \text{si } \text{tam}(s_1) < n \leq \text{tam}(s) \\ \epsilon & \text{en otro caso} \end{cases}$$

Nota: La función línea devuelve el texto de una línea concreta de un código While, o bien la cadena vacía si el número de línea no es válido.



Función salto...

Definición 6: Función salto...

Definimos la función **salto** : $\text{Cód_While} \times \mathbb{N} \rightarrow \mathbb{N}$ como:

1. Si en G_2 se cumple que $\langle \text{asignación} \rangle \Rightarrow^* s$, entonces: $\text{salto}(s, n) = 0$
2. Si $s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od}$, con $i \in \Sigma_d^+$ y $s_1 \in \text{Cód_While}$, entonces:

$$\text{salto}(s, n) = \begin{cases} \text{tam}(s) + 1 & \text{si } n=1 \\ 1 & \text{si } n=\text{tam}(s) \\ \text{salto}(s_1, n-1) + 1 & \text{si } (1 < n < \text{tam}(s)) \wedge (\text{salto}(s_1, n-1) \neq 0) \\ 0 & \text{en otro caso} \end{cases}$$



... Función salto

Definición 6: .. Función salto

3. Si $s = s_1 ; s_2$, con $s_1, s_2 \in \text{Cód_While}$, entonces:

$$\text{salto}(s, n) = \begin{cases} \text{salto}(s_1, n) & 0 < n \leq \text{tam}(s_1) \\ \text{salto}(s_2, n - \text{tam}(s_1)) + \text{tam}(s_1) & \text{si } (\text{tam}(s_1) < n \leq \text{tam}(s)) \wedge (\text{salto}(s_2, n - \text{tam}(s_1)) \neq 0) \\ 0 & \text{en otro caso} \end{cases}$$

Nota: La función salto devuelve **el número de línea no consecutiva** al que se podría saltar desde una **línea dada** (N) de un **código While** (Cód_While), o bien **cero** si desde la línea especificada no se puede saltar a una línea no consecutiva.



Preguntas

Definición, tamaño, línea y salto



Preguntas

1. $\zeta(1, 2, X_3 := 0) \in \text{WHILE?}$

No. Un programa con dos variables en total no puede tener la variable X_3 en su código.

2. ¿Un programa WHILE puede tener infinitas variables?

No. Por definición, la segunda componente de la terna que define un programa WHILE, que indica cuantas variables tiene el programa, es un natural, por lo que el número de variables siempre es finito.

3. ¿Si $s = s_1 ; s_2$ con $s_1, s_2 \in \text{Cód_WHILE}$ entonces $\text{tam}(s) > \text{tam}(s_1)$?

Sí. Puesto que un código siempre tiene al menos tamaño uno (según definición por la gramática), y el tamaño de una secuencia es la suma de los tamaños de sus códigos.

4. $\zeta \text{ tam}(s) > n \rightarrow \text{línea}(s, n) = 0$?

No. La función línea nunca devuelve cero. Devuelve el texto de una línea concreta de un código While, o bien la cadena vacía si el número de línea no es válido.

5. $\zeta n \in \mathbb{N} - \{0\} \wedge \exists s \in \text{Cód_While} \mid \text{salto}(s, n) = n$?

No. La función salto, por definición, nunca devuelve la misma línea de la que parte.



Configuración, Conjunto de configuraciones

Definición 7: Configuración, Conjunto de configuraciones

Sea $Q \in \text{WHILE}$, con $Q=(n,p,s)$. Definimos el **conjunto de configuraciones** de Q , notado como C_Q como:

$$C_Q = \{ \underline{c} \in N^{p+1} \mid \underline{c} = (m, \underline{x}) \text{ con } 1 \leq m \leq \text{tam}(Q)+1 \text{ y } \underline{x} \in N^p \}$$

Diremos que $\underline{c}$ es una **configuración** de Q sii $c \in C_Q$.

Nota: La configuración nos indica la posición en el código while y el estado de las variables del programa.



Configuración inicial, terminal

Definición 8: Configuración inicial

Sea $Q \in \text{WHILE}$, con $Q=(n,p,s)$, y sea $\underline{c} \in C_Q$, con $\underline{c} = (m, \underline{x})$. Diremos que $\underline{c}$ es **configuración inicial** de Q sii

$$m = 1 \wedge x_{n+1} = \dots = x_p = 0$$

Nota: Los valores de las variables que no son de entrada, son ceros.

Definición 9: Configuración terminal, Configuración no terminal

Sea $Q \in \text{WHILE}$, y sea $\underline{c} \in C_Q$, con $\underline{c} = (m, \underline{x})$.

- Diremos que $\underline{c}$ es **configuración terminal** de Q sii $m = \text{tam}(Q) + 1$.
- En caso contrario decimos que es una **configuración no terminal**.



Transitar directamente

Definición 10: Transitar directamente

Sea $Q \in \text{WHILE}$, con $Q=(n,p,s)$. Sean $\underline{c_1}, \underline{c_2} \in C_Q$, con $\underline{c_1} = (m, \underline{x})$ y $\underline{c_2} = (t, \underline{z})$. Diremos que Q **transita directamente** de $\underline{c_1}$ a $\underline{c_2}$, notado como $\underline{c_1} \vdash \underline{c_2}$, sii:

Asignaciones

linea(s,m) = $X_i := 0$ con $i \in \Sigma_d^+$

$\Rightarrow t = m+1 \wedge z_i = 0 \wedge (z_r = x_r \forall r \neq i)$

linea(s,m) = $X_i := X_j$ con $i \in \Sigma_d^+$

$\Rightarrow t = m+1 \wedge z_i = z_j \wedge (z_r = x_r \forall r \neq i)$

linea(s,m) = $X_i := X_j + 1$ con $i \in \Sigma_d^+$

$\Rightarrow t = m+1 \wedge z_i = z_j + 1 \wedge (z_r = x_r \forall r \neq i)$

linea(s,m) = $X_i := X_j - 1$ con $i \in \Sigma_d^+$

$\Rightarrow t = m+1 \wedge z_i = z_j - 1 \wedge (z_r = x_r \forall r \neq i)$

While/od

linea(s,m) = while $X_i \neq 0$ do con $i \in \Sigma_d^+$

$\Rightarrow (x_i \neq 0 \Rightarrow t = m+1) \wedge (x_i = 0 \Rightarrow t = \text{salto}(s,m)) \wedge (\underline{z} = \underline{x})$

linea(s,m) = od

$\Rightarrow t = \text{salto}(s,m) \wedge (\underline{z} = \underline{x})$



Ejemplo 1

Sea $Q = (1, 2, s)$ con s :

```
 $X_2 := X_1;$   
while  $X_2 \neq 0$  do  
   $X_1 := X_1 + 1;$   
   $X_2 := X_2 - 1$   
od
```

¡¡ Recuerda !!

Configuraciones $\underline{c} = (m, \underline{x})$

Inicial: $m=1 \wedge x_{n+1} = \dots x_p = 0$

Terminal: sii $m = \text{tam}(Q) + 1$

- El vector $(1,1,1)$ es una configuración de Q , es decir, $(1,1,1) \in C_Q$.
 - No es una configuración inicial, ya que la tercera componente no es cero.
 - No es terminal, puesto que la primera componente no es seis. ($\text{tam}(Q) + 1 = 6$)
 - $(1,1,1) \vdash (2,1,1)$, es decir, Q transita directamente de $(1,1,1)$ a $(2,1,1)$
- El vector $(0,1,1)$ no es una configuración de Q ,
 - $(0,1,1) \notin C_Q$, ya que $0 \notin [1, 6]$.
 - no transita directamente a ninguna configuración, ya que no es una configuración
- El vector $(1,10,0) \in C_Q$,
Siendo además configuración inicial. $(1,10,0) \vdash (2,10,10)$
- El vector $(6,6,6) \in C_Q$,
Siendo además configuración terminal. Al ser terminal, no transita directamente a ninguna otra.



Preguntas Configuraciones y Trancisiones



Preguntas

1. ¿Algún programa WHILE tiene un número finito de configuraciones iniciales?

Para que las configuraciones sean infinitas dependen del número de línea y del valor de las variables de entrada. Como el número de líneas es finito, solo depende de las variables de entrada. Si tiene una variable de entrada ya sería infinita. Solo sería finita si no tuviera variables de entrada.

2. Una configuración de un programa WHILE puede ser a la vez inicial y terminal?

No. Ya que las configuraciones tienen como primera componente el número de línea en el que está el programa. Las configuraciones iniciales tienen un valor de 1, mientras que las finales tienen uno más del tamaño del programa. Como el programa no puede ser vacío tendrá que tener al menos valor 2.

3. ¿ $c_1 \vdash c_2 \Rightarrow c_1 \neq c_2$?

Sí. El número de línea cambia de una configuración a su siguiente, mayor o menor, pero nunca igual.



Función siguiente (SIG)

Definición 11: Función siguiente (SIG)

Sea $Q \in \text{WHILE}$, con $Q=(n,p,s)$. Sea C_Q el conjunto de configuraciones de Q . Definimos la función siguiente de Q , notada SIG_Q , como: $SIG_Q : N^{p+1} \rightarrow N^{p+1}$

$$SIG_Q(\underline{c}) = \begin{cases} \underline{c}' & \text{si } \underline{c} \vdash \underline{c}' \text{ con } \underline{c}, \underline{c}' \in C_Q \\ \underline{c} & \text{si } \nexists \underline{c}' \in C_Q \mid \underline{c} \vdash \underline{c}' \text{ con } \underline{c} \in C_Q \\ \underline{c} & \text{si } \underline{c} \notin C_Q \end{cases}$$

Nota: Amplía el concepto “transitar directamente” dándonos una función total. Simplemente me quedo en la misma configuración si no hay una configuración siguiente o no es válida.



Función cálculo (CAL)

Definición 12: Función cálculo (CAL)

Sea $Q \in \text{WHILE}$, con $Q=(n,p,s)$. Definimos la función cálculo de Q , notada CAL_Q , como $N^{n+1} \rightarrow N^{p+1}$.

$$CAL_Q(\underline{x}, i) = \begin{cases} (1, \underline{x}, \underline{0}) & \text{si } i = 0 \\ SIG_Q(CAL_Q(\underline{x}, i - 1)) & \text{si } i > 0 \end{cases}$$

Nota: $i \in N$, $\underline{x} \in N^n$ y $\underline{0} \in N^{p-n}$



Ejemplo 1

Sea $Q = (1, 2, s)$ con s :

```
 $X_2 := X_1;$   
while  $X_2 \neq 0$  do  
   $X_1 := X_1 + 1;$   
   $X_2 := X_2 - 1$   
od
```

$SIG_Q(1,1,1)=(2,1,1)$
 $SIG_Q(0,1,1)=(0,1,1)$
 $SIG_Q(7,1,1)=(7,1,1)$
 $SIG_Q(1,10,0)=(2,10,10)$
 $SIG_Q(6,6,6)=(6,6,6)$

$$SIG_Q(\underline{c}) = \begin{cases} \underline{c}' & \text{si } \underline{c} \vdash \underline{c}' \text{ con } \underline{c}, \underline{c}' \in C_Q \\ \underline{c} & \text{si } \nexists \underline{c}' \in C_Q \mid \underline{c} \vdash \underline{c}' \text{ con } \underline{c} \in C_Q \\ \underline{c} & \text{si } \underline{c} \notin C_Q \end{cases}$$



Ejemplo 1

Sea $Q = (1, 2, s)$ con s :

$X_2 := X_1;$

while $X_2 \neq 0$ do

$X_1 := X_1 + 1;$

$X_2 := X_2 - 1$

od

$CAL_Q(2,4) =$

$SIG_Q(CAL_Q(2,3)) =$

$SIG_Q(SIG_Q(CAL_Q(2,2))) =$

$SIG_Q(SIG_Q(SIG_Q(CAL_Q(2,1)))) =$

$SIG_Q(SIG_Q(SIG_Q(SIG_Q(CAL_Q(2,0))))) =$

$SIG_Q(SIG_Q(SIG_Q(SIG_Q(1,2,0)))) = (1,2,0) \mid - (2,2,2)$

$SIG_Q(SIG_Q(SIG_Q(2,2,2))) = (2,2,2) \mid - (3,2,2)$

$SIG_Q(SIG_Q(3,2,2)) = (3,2,2) \mid - (4,3,2)$

$SIG_Q(4,3,2) = (5,3,1) (4,3,2) \mid - (5,3,1)$

$$CAL_Q(\underline{x}, i) = \begin{cases} (1, \underline{x}, \underline{0}) & \text{si } i = 0 \\ SIG_Q(CAL_Q(\underline{x}, i - 1)) & \text{si } i > 0 \end{cases}$$



Función complejidad temporal (T)

Definición 13: Función complejidad temporal (T)

Sea $Q \in \text{WHILE}$, con $Q=(n,p,s)$.

Definimos la función complejidad de Q , notada T_Q , como $T_Q : N^n \rightarrow N$.

$$T_Q(\underline{x}) = \begin{cases} \text{minimo}(A) & \text{si } A \neq \emptyset \\ \uparrow & \text{si } A = \emptyset \end{cases}$$

donde $A = \{j \in N \mid \pi_1^{p+1}(\text{CAL}_Q(\underline{x}, j)) = \text{tam}(Q) + 1\}$

*Nota*₁: $\pi_1^3(11, 22, 33) = 11$. En nuestro caso el valor de la línea.

*Nota*₂: El Conjunto A está formado por todos los valores j que finalizan el programa.

*Nota*₃: Establece el número de la iteración que termina si existe, en caso contrario no termina.



Ejemplo 1

Sea $Q = (1, 2, s)$ con s :

$X_2 := X_1$;

while $X_2 \neq 0$ do

$X_1 := X_1 + 1$;

$X_2 := X_2 - 1$

od

$CAL_Q(2, j) = 6$

$SIG_Q(CAL_Q(2, 10)) =$

$SIG_Q(SIG_Q(SIG_Q(SIG_Q(SIG_Q(SIG_Q(SIG_Q(SIG_Q(SIG_Q(CAL_Q(2, 0) \dots)))$

$\dots$

$SIG_Q(2, 4, 0) =$

$(6, 4, 0)$

SIG	Línea	X_1	X_2
-	1	2	0
1	2	2	2
2	3	2	2
3	4	3	2
4	5	3	1
5	2	3	1
6	3	3	1
7	4	4	1
8	5	4	0
9	2	4	0
10	6	4	0

Función complejidad temporal al vector (2)
tenemos: $T_Q(2) = 10$ puesto que:
 $\text{mínimo } \{j \in N \mid \pi_1^3(CAL_Q(2, j)) = 6\} = 10$



Función calculada (F)

Definición 14: Función calculada (F)

Sea $Q \in \text{WHILE}$, con $Q=(n,p,s)$.

Definimos la función calculada de Q , notada F_Q , como $F_Q : N^n \rightarrow N$.

$$F_Q(\underline{x}) = \pi_2^{p+1} (CAL_Q(\underline{x}, T_Q(\underline{x})))$$

*Nota*₁: $\pi_2^3(11, 22, 33) = 22$. En nuestro caso el valor de X_1 .

*Nota*₂: Aplicamos la función CAL_Q el mínimo número de veces que necesitamos para el programa termine.

Ejemplo: $(1,2,0) \vdash (2,2,2) \vdash (3,2,2) \vdash (4,3,2) \vdash (5,3,1) \vdash (2,3,1) \vdash (3,3,1) \vdash (4,4,1) \vdash (5,4,0) \vdash (2,4,0) \vdash (6,4,0)$.
11 configuraciones, 10 transiciones



Preguntas

Función Calculada y complejidad Temporal



Preguntas

1. $\forall Q \in \text{WHILE} \forall i \in \mathbb{N}, \text{CAL}_Q(x, i) = \text{CAL}_Q(x, i + 1) \Rightarrow T_Q(x) \leq i$?

Sí. Si dos configuraciones consecutivas son iguales significa que en ese número de pasos, o menos, el programa ya ha terminado, por lo que acota la función complejidad temporal.

2. $\exists Q = (n, p, s) \in \text{WHILE} \mid T_Q(x) = 0 \text{ con } x \in \mathbb{N}_N$?

No. La función complejidad temporal o bien diverge, o bien devuelve un natural mayor que cero, pero nunca puede devolver cero ya que todo programa ha de tener alguna sentencia.

3. ¿Si Q es un programa WHILE entonces F_Q diverge si y sólo si T_Q diverge?

Sí. Dado que la función complejidad temporal nos indica en cuantos pasos acaba el programa, si ella es total la función calculada también, y si ella diverge para algún valor, para ese valor la función calculada también diverge.



Ejemplo 1



Ejemplo 1

Sea $Q = (1, 2, s)$ con s:

```
 $X_2 := X_1;$   
while  $X_2 \neq 0$  do  
   $X_1 := X_1 + 1;$   
   $X_2 := X_2 - 1$   
od
```

$$SIG_Q(\underline{c}) = \begin{cases} (0, b, c) & \text{si } a = 0 \text{ No es una configuración } \underline{c} \notin C_0 \\ (2, b, b) & \text{si } a = 1 \text{ Asignación } X_2 := X_1; \\ (3, b, c) & \text{si } a = 2 \wedge c \neq 0 \text{ Entra bucle } c = 0; \\ (6, b, c) & \text{si } a = 2 \wedge c = 0 \text{ Siguiente fuera bucle } c = 0 \\ (4, b+1, c) & \text{si } a = 3 \text{ } X_1 := X_1 + 1 \\ (5, b, c-1) & \text{si } a = 4 \text{ } X_2 := X_2 - 1 \\ (2, b, c) & \text{si } a = 5 \text{ od} \\ (a, b, c) & \text{si } a > 5 \text{ si } \nexists \underline{c}' \in C_Q \mid c \vdash \underline{c}' \text{ con } \underline{c} \in C_Q \end{cases}$$



Ejemplo

Si comenzamos el cómputo con la configuración inicial $(1, n, 0)$, con $n > 0$, tenemos:

$(1, n, 0) \vdash (2, n, n) \vdash (3, n, n) \vdash (4, n+1, n) \vdash (5, n+1, n-1) \vdash (2, n+1, n-1) \vdash (3, n+1, n-1) \vdash$
 $(4, n+2, n-1) \vdash (5, n+2, n-2) \vdash (2, n+2, n-2) \vdash (3, n+2, n-2) \vdash$
 $\dots$
 $(4, 2n-1, 2) \vdash (5, 2n-1, 1) \vdash (2, 2n-1, 1) \vdash (3, 2n-1, 1) \vdash (4, 2n, 1) \vdash (5, 2n, 0) \vdash (2, 2n, 0) \vdash (6, 2n, 0)$

y para $n = 0$ tenemos: $(1, 0, 0) \vdash (2, 0, 0) \vdash (6, 0, 0)$

Por tanto: $T_Q(n) = 4n+2 \ \forall n \in N \ F_Q(n) = 2n \ \forall n \in N$.

Tampoco hay una forma genérica de saber cuanto vale la función complejidad temporal de un programa para un argumento dado.

- Para evaluarla debemos ejecutar el programa con dicho argumento como entrada hasta que se alcance (si es el caso) una configuración terminal, siendo el valor de la función complejidad temporal el número de transiciones directas realizadas durante el cómputo.
- En el caso de que no se pueda alcanzar una configuración terminal tanto la función complejidad



Ejemplo 2



Ejemplo 2

Sea $Q = (1, 1, s)$ con s :

```
 $X_1 := X_1 + 1;$   
while  $X_1 \neq 0$  do  
   $X_1 := X_1$   
od
```

$$\text{WHILE} = \{ (n, p, s) \in \mathbb{N} \times \mathbb{N} \times \text{Cód_While} \mid p \geq n \wedge p > 0 \wedge (X_m \succ s \text{ con } m \in \Sigma_d^+ \Rightarrow m \leq p) \}$$

$Q \in \text{WHILE}$, es decir, es un programa WHILE, ya que se cumple que:

$Q \in \mathbb{N} \times \mathbb{N} \times \text{Cód_While} \wedge$

$1 \geq 1 \wedge (p \geq n)$

$1 > 0 \wedge p > 0$

$(X_m \succ s \text{ con } m \in \Sigma_d^+ \Rightarrow m \leq 1)$

El hecho de que la tercera componente de Q , es decir, la cadena s , pertenezca a Cód_While significa que s es un código en lenguaje WHILE sintácticamente correcto, cumpliéndose que $\text{tam}(Q) = \text{tam}(s) = 4$.



Ejemplo 2

- El vector $(2,0)$ es una configuración de Q , es decir, $(2,0) \in C_Q$
 - No es una configuración inicial, ya que la primera componente no es uno
 - Ni terminal, puesto que la primera componente no es cinco
 - Se cumple que $(2,0) \vdash (5,0)$, Q transita directamente de $(2,0)$ a $(5,0)$
- El vector $(0,1)$ no es una configuración de Q
 - $(0,1) \notin C_Q$, ya que $0 \notin [1, 5]$.
 - no transita directamente a ninguna configuración al no serlo él
- El vector $(6,1)$ no es una configuración de Q
 - $(6,1) \notin C_Q$, ya que $6 \notin [1, 5]$.
 - no transita directamente a ninguna configuración al no serlo él
- El vector $(1,10) \in C_Q$, siendo además configuración inicial.
 - Se cumple que $(1,10) \vdash (2,11)$, Q transita directamente de $(1,10)$ a $(2,11)$
- La configuración $(5,5)$, al ser terminal, no transita directamente a ninguna otra.

Sea $Q = (1, 1, s)$ con s :

```

$$\begin{aligned} &X_1 := X_1 + 1; \\ &\text{while } X_1 \neq 0 \text{ do} \\ &\quad X_1 := X_1 \\ &\text{od} \end{aligned}$$

```



Ejemplo 2

Si aplicamos la función siguiente a estos vectores tenemos:

$SIG_Q(2,0)=(5,0)$. fin bucle

$SIG_Q(0,1)=(0,1)$. $c \notin C_Q$

$SIG_Q(6,1)=(6,1)$. $c \notin C_Q$

$SIG_Q(1,10)=(2,11)$. entra while

$SIG_Q(5,5)=(5,5)$. $\nexists \underline{c}' \in C_Q \mid \underline{c} \vdash \underline{c}'$ con $\underline{c} \in C_Q$

Si aplicamos la función cálculo al vector (2,4) tenemos:

$CAL_Q(2,4) =$

$SIG_Q(CAL_Q(2,3)) =$

$SIG_Q(SIG_Q(CAL_Q(2,2))) =$

$SIG_Q(SIG_Q(SIG_Q(CAL_Q(2,1)))) =$

$SIG_Q(SIG_Q(SIG_Q(SIG_Q(CAL_Q(2,0))))) =$

$SIG_Q(SIG_Q(SIG_Q(SIG_Q(1,2)))) =$

$SIG_Q(SIG_Q(SIG_Q(2,3))) =$

$SIG_Q(SIG_Q(3,3)) =$

$SIG_Q(4,3) = (2,3)$

Sea $Q = (1, 1, s)$ con s:

$X_1 := X_1 + 1;$

while $X_1 \neq 0$ do

$X_1 := X_1$

od



Ejemplo 2

Si aplicamos la función complejidad temporal al vector (2) tenemos: $T_Q(2) = \uparrow$

Debido a que hay un bucle no podemos llegar a la sentencia siguiente al bucle, y por tanto:

$$\{j \in \mathbb{N} \mid \pi_1^2(CAL_Q(2, j)) = 5\} = \emptyset$$

Si aplicamos la función calculada al mismo vector tenemos: $F_Q(2) = \uparrow$

```
Sea Q = (1, 1, s) con s:  
  X1 := X1 + 1;  
  while X1 ≠ 0 do  
    X1 := X1  
  od
```



Ejemplo 2

$$SIG_Q(\underline{c}) = \begin{cases} (0, b) & \text{si } a = 0 \\ (2, b + 1) & \text{si } a = 1 \\ (3, b) & \text{si } a = 2 \wedge b \neq 0 \\ (5, b) & \text{si } a = 2 \wedge b = 0 \\ (4, b) & \text{si } a = 3 \\ (2, b) & \text{si } a = 4 \\ (a, b) & \text{si } a > 4 \end{cases}$$

Si comenzamos el cómputo con la configuración inicial $(1, n)$ tenemos:

$(1, n) \vdash (2, n+1) \vdash (3, n+1) \vdash (4, n+1) \vdash (2, n+1) \vdash \dots$ y puesto que la configuración $(2, n+1)$ aparece dos veces tenemos un bucle, y por tanto:

$$T_Q(n) = \uparrow \forall \in N \text{ y } F_Q(n) = \uparrow \forall \in N$$

Sea $Q = (1, 1, s)$ con s :

```
 $X_1 := X_1 + 1;$   
while  $X_1 \neq 0$  do  
   $X_1 := X_1$   
od
```



Ejemplos 3,4 y 5

Ejemplo 3: Sea el programa WHILE $Q = (1, 1, s)$ con s :

$F_Q(n) = n + 2 \forall n \in \mathbb{N}$, es la función **sumados**.

Ejemplo 4: Sea el programa WHILE $Q = (2, 2, s)$ con s :

$F_Q(n, m) = n + m \forall n, m \in \mathbb{N}$, es la función **suma**.

Ejemplo 5: Sea el programa WHILE $Q = (1, 1, s)$ con s :

$$F_Q(n) = \begin{cases} 0 & \text{si } x = 0 \\ \uparrow & \text{si } x > 0 \end{cases}$$

$F_Q(n)$ es la función **cero**, parcial.

```
X1 := X1 + 1;  
X1 := X1 + 1
```

```
while X2 ≠ 0 do  
  X1 := X1 + 1;  
  X2 := X2 - 1  
od
```

```
while X1 ≠ 0 do  
  X1 := X1;  
od
```



Computabilidad



Función WHILE-computable, $F(WHILE)$

Definición 15: Función WHILE-computable, $F(WHILE)$

Sea la función $f: N^n \rightarrow N$ con $n \geq 0$.

Diremos que f es una función WHILE-computable si existe un programa WHILE Q tal que $F_Q = f$.

- En ese caso diremos que dicho programa WHILE-computa la función f .
- Al conjunto de **todas las funciones WHILE-computables** lo llamaremos $F(WHILE)$.
- Al conjunto de todas las funciones WHILE-computables $f: N^n \rightarrow N$ lo llamaremos $F^n(WHILE)$.

Definición 16: T-WHILE

$T-WHILE = \{f \in F(WHILE) \mid f \text{ es total} \}$



Predicado WHILE-decidible, $\text{PRED}(\text{T-WHILE})$

Definición 17: P es un predicado WHILE-decidible, $\text{PRED}(\text{T-WHILE})$

$$\text{PRED}(\text{T-WHILE}) = \{ P_f \mid f \in \text{T-WHILE} \}$$

Diremos que P es un predicado WHILE-decidible sii $P \in \text{PRED}(\text{T-WHILE})$.

Definición 18: P es un predicado WHILE-enumerable, $\text{PRED}(\text{WHILE})$

$$\text{PRED}(\text{WHILE}) = \{ P_f \mid f \in \text{WHILE} \}$$

Diremos que P es un predicado WHILE-enumerable $P \in \text{PRED}(\text{WHILE})$.

$\text{PRED}(\text{T-WHILE}) \subset \text{PRED}(\text{WHILE})$ ya que $\text{T-WHILE} \subset \text{F(WHILE)}$.



Ejercicios



Ejercicios 1 - 4

Ejercicio 1: Demostrar constructivamente que la **función cero** es WHILE-computable, siendo $\text{cero}: N^0 \rightarrow N$, donde $\text{cero}()=0$.

Sea el programa WHILE $Q = (0, 1, s)$ con $s: X_1 := 0$. $F_Q = \text{cero}$

Ejercicio 2: Demostrar constructivamente que la **función identidad** es WHILE-computable, siendo $\text{identidad}: N \rightarrow N$, donde $\text{identidad}(n) = n$.

Sea el programa WHILE $Q = (1, 1, s)$ con $s: X_1 := X_1$. $F_Q = \text{identidad}$

Ejercicio 3: Demostrar constructivamente que la **función sucesor** es WHILE-computable, siendo $\text{sucesor}: N \rightarrow N$, donde $\text{sucesor}(n) = n+1$.

Sea el programa WHILE $Q = (1, 1, s)$ con $s: X_1 := X_1 + 1$. $F_Q = \text{sucesor}$

Ejercicio 4: Demostrar constructivamente que la **función predecesor** es WHILE-computable, siendo $\text{predecesor}: N \rightarrow N$, donde $\text{predecesor}(n) = \max\{n-1, 0\}$.

Sea el programa WHILE $Q = (1, 1, s)$ con $s: X_1 := X_1 - 1$. $F_Q = \text{predecesor}$



Ejercicios 5-6

Ejercicio 5: Demostrar constructivamente que la **función suma** es WHILE-computable, siendo $\text{suma}: \mathbb{N}^2 \rightarrow \mathbb{N}$, donde $\text{suma}(n,m)=n+m$.

Sea el programa WHILE $Q = (2, 2, s)$ con s:

while $X_2 \neq 0$ do

$X_1 := X_1 + 1;$

$X_2 := X_2 - 1$

od

$F_Q = \text{suma}$

Ejercicio 6: Demostrar constructivamente que la **función resta** natural es WHILE-computable, siendo $\text{resta}: \mathbb{N}^2 \rightarrow \mathbb{N}$, donde $\text{resta}(n,m) = \max(n-m, 0)$. Sea el programa WHILE $Q = (2, 2, s)$ con s:

while $X_2 \neq 0$ do

$X_1 := X_1 - 1;$

$X_2 := X_2 - 1$

od

$F_Q = \text{resta}$



Ejercicio 7

Ejercicio 7: Demostrar constructivamente que la *función csigno* es WHILE-computable, siendo $\text{csigno}: \mathbb{N} \rightarrow \mathbb{N}$, donde $\text{csigno}(m) = 1$, si $m=0$ y $\text{csigno}(m) = 0$, si $m>0$.

Sea el programa WHILE $Q = (1, 2, s)$ con s :

```
 $X_2 := X_2 + 1;$   
while  $X_1 \neq 0$  do  
   $X_2 := 0;$   
   $X_1 := 0$   
od  
 $X_1 := X_2$   
 $F_Q = \text{csigno}$ 
```



Ejercicio 8

Ejercicio 8: Hemos definido la **función cero** mediante una función total y una función parcial.

- **Función Total** $[f_1(x) : N \rightarrow N]$: $Q = (1, 1, s)$ con $s: X_1 := 0$
- **Función Parcial** $[f_2(x) : N \rightarrow N]$: $Q = (1, 1, s)$ con $s: \text{while } X_1 \neq 0 \text{ do } X_1 := X_1; \text{od}$

$$F_Q(n) = \begin{cases} 0 & \text{si } x = 0 \\ \uparrow & \text{si } x > 0 \end{cases}$$

Si definimos el Predicado $P(X) = \text{cero}$, que daría verdadero si X es cero.

- **Función Total:** P_{f_1} es decidable ya que $f_1 \in \text{T-WHILE}$.
- **Función Parcial** P_{f_2} es enumerable ya que $f_2 \in \text{WHILE}$.



Codificación de programas WHILE



Codificación de Cantor y Gödel

Para realizar la codificación de programas WHILE es necesario definir las codificaciones de **Cantor** y **Gödel**.

La codificación (o gödelización) de programas WHILE consiste en definir una biyección recursiva entre el conjunto de los programas WHILE y los naturales, la cual asignará biunívocamente a cada programa un número.

1. Para ello primero veremos cómo codificar sentencias (función **codi**),
2. después cómo codificar códigos (función **Codi**),
3. y finalmente cómo codificar programas (función **CODI**).



Codificación de Cantor

Codificación de Cantor, también llamada **cantorización**, de tamaño dos, notada σ_1^2 , como:

$$\sigma_1^2: N^2 \rightarrow N$$

$$\sigma_1^2(x, y) = (x+y)(x+y+1)/2 + y$$

Decodificación de Cantor, también llamada **decantorización**:

- **La segunda componente**, $\sigma_{2,2}^1: N \rightarrow N$
 $\sigma_{2,2}^1(x) = x - \sigma_1^2(d(x), 0)$
- **La primera componente**, $\sigma_{2,1}^1: N \rightarrow N$
 $\sigma_{2,1}^1(x) = d(x) - \sigma_{2,2}^1(x)$
- donde $d(x) = \min \{ t \in N \mid \sigma_1^2(t, 0) > x \} - 1 = \min \{ t \in N \mid t(t+1)/2 > x \} - 1$.

Codificación de Cantor de tamaño $k+1$, con $k > 1$, como:

$$\sigma_1^{k+1}: N^{k+1} \rightarrow N$$

$$\sigma_1^{k+1}(x_1, x_2, \dots, x_{k+1}) = \sigma_1^2(x_1, \sigma_1^k(x_2, \dots, x_{k+1}))$$

Decodificación de Cantor, también llamada **decantorización**:

$$\sigma_{k+1,i}^1: N \rightarrow N$$

$$\sigma_{k+1,i}^1(x) = \begin{cases} \sigma_{2,1}^1(x) & : i = 1 \\ \sigma_{k,i-1}^1(\sigma_{2,2}^1(x)) & : 2 \leq i \leq k+1 \end{cases}$$



Ejemplos Codificación de Cantor

Codificación de Cantor, de tamaño dos, notada σ_1^2 , como:

$$\sigma_1^2: N^2 \rightarrow N$$

$$\sigma_1^2(x, y) = (x+y)(x+y+1)/2 + y$$

$$\sigma_1^2(0, 0) = (0+0)(0+0+1)/2 + 0 = 0$$

$$\sigma_1^2(1, 0) = (1+0)(1+0+1)/2 + 0 = 1$$

$$\sigma_1^2(0, 1) = (0+1)(0+1+1)/2 + 1 = 2$$

$$\sigma_1^2(2, 0) = (2+0)(2+0+1)/2 + 0 = 3$$

$$\sigma_1^2(1, 1) = (1+1)(1+1+1)/2 + 1 = 4$$

Codificación de Cantor de tamaño $k+1$, con $k > 1$, como:

$$\sigma_1^{k+1}: N^{k+1} \rightarrow N$$

$$\sigma_1^{k+1}(x_1, x_2, \dots, x_{k+1}) = \sigma_1^2(x_1, \sigma_1^k(x_2, \dots, x_{k+1}))$$

$$\sigma_1^3(0, 0, 0) = \sigma_1^2(0, \sigma_1^2(0, 0)) = \sigma_1^2(0, 0) = 0.$$

$$\sigma_1^3(1, 0, 0) = \sigma_1^2(1, \sigma_1^2(0, 0)) = \sigma_1^2(1, 0) = 1.$$

$$\sigma_1^3(0, 1, 0) = \sigma_1^2(0, \sigma_1^2(1, 0)) = \sigma_1^2(0, 1) = 2.$$

$$\sigma_1^3(2, 0, 0) = \sigma_1^2(2, \sigma_1^2(0, 0)) = \sigma_1^2(2, 0) = 3.$$

$$\sigma_1^3(1, 1, 0) = \sigma_1^2(1, \sigma_1^2(1, 0)) = \sigma_1^2(1, 1) = 4.$$



Ejemplos Decodificación de Cantor

Decodificación de Cantor:

- **La segunda componente**, $\sigma_{2,2}^1: \mathbb{N} \rightarrow \mathbb{N}$
 $\sigma_{2,2}^1(x) = x - \sigma_1^2(d(x), 0)$
- **La primera componente**, $\sigma_{2,1}^1: \mathbb{N} \rightarrow \mathbb{N}$
 $\sigma_{2,1}^1(x) = d(x) - \sigma_{2,2}^1(x)$
- donde $\mathbf{d(x)} = \min \{ t \in \mathbb{N} \mid \sigma_1^2(t, 0) > x \} - 1 = \min \{ t \in \mathbb{N} \mid t(t+1)/2 > x \} - 1$.

1 $\rightarrow$ (1,0)

$$\sigma_{2,2}^1(1) = 1 - \sigma_1^2(d(1), 0) = 1 - \sigma_1^2(1, 0) = 1 - 1 = 0$$

$$\sigma_{2,1}^1(1) = d(1) - \sigma_{2,2}^1(1) = 1 - 0 = 1$$

2 $\rightarrow$ (0,1)

$$\sigma_{2,2}^1(2) = 2 - \sigma_1^2(d(2), 0) = 2 - \sigma_1^2(1, 0) = 2 - 1 = 1$$

$$\sigma_{2,1}^1(2) = d(2) - \sigma_{2,2}^1(2) = 1 - 1 = 0$$

Decodificación de Cantor, también llamada **decantorización**:

$$\sigma_{k+1,i}^1: \mathbb{N} \rightarrow \mathbb{N}$$

$$\sigma_{k+1,i}^1(x) = \begin{cases} \sigma_{2,1}^1(x) & : i = 1 \\ \sigma_{k+1,i}^1(\sigma_{2,2}^1(x)) & : 2 \leq i \leq k+1 \end{cases}$$

$$\sigma_1^3(x, y, z) = \sigma_1^2(x, \sigma_1^2(y, z))$$

$$\sigma_{3,1}^1(w) = \sigma_{2,1}^1(w)$$

$$\sigma_{3,2}^1(w) = \sigma_{2,1}^1(\sigma_{2,2}^1(w))$$

$$\sigma_{3,3}^1(w) = \sigma_{2,2}^1(\sigma_{2,2}^1(w))$$



Codificación de Gödel

Codificación de Gödel, gödelización (god):

$god: N^* \rightarrow N$

$$god(\underline{x}) = \begin{cases} 0 & \text{si } L(\underline{x}) = 0 \\ \sigma_1^2(L(\underline{x}) - 1, \sigma_1^{L(\underline{x})}(\underline{x})) + 1 & \text{si } L(\underline{x}) \neq 0 \end{cases}$$

con $L: N^* \rightarrow N$

$L(\epsilon) = 0$

$L(\underline{x}) = m \quad \forall (\underline{x}) \in N^m \text{ (con } m > 0)$

Decodificación de Gödel, degödelización (degod):

$degod: N^2 \rightarrow N$

$$degod(z, k) = \begin{cases} 0 & \text{si } z = 0 \vee k > l(z) \\ \sigma_{l(z), k}^1(\sigma_{2, 2}^1(z - 1)) & \text{resto} \end{cases}$$

con $l: N \rightarrow N$

$l(0) = 0$

$l(z) = \sigma_{2, 1}^1(z - 1) + 1 \text{ (para } z > 0)$



Ejemplos Codificación de Gödel

Codificación de Gödel, gödelización (god):

$god: N^* \rightarrow N$

$$god(\underline{x}) = \begin{cases} 0 & \text{si } L(\underline{x}) = 0 \\ \sigma_1^2(L(\underline{x}) - 1, \sigma_1^{L(\underline{x})}(\underline{x})) + 1 & \text{si } L(\underline{x}) \neq 0 \end{cases}$$

con $L: N^* \rightarrow N$

$L(\epsilon) = 0$

$L(\underline{x}) = m \quad \forall (\underline{x}) \in N^m \text{ (con } m > 0)$

$$god(0) = \sigma_1^2(1 - 1, \sigma_1^1(0)) + 1 = \sigma_1^2(0, 0) + 1 = 0 + 1 = 1$$

$$god(1) = \sigma_1^2(1 - 1, \sigma_1^1(1)) + 1 = \sigma_1^2(0, 1) + 1 = 2 + 1 = 3$$

$$god(2) = \sigma_1^2(1 - 1, \sigma_1^1(2)) + 1 = \sigma_1^2(0, 2) + 1 = 5 + 1 = 6$$

$$god(0, 0) = \sigma_1^2(2 - 1, \sigma_1^2(0, 0)) + 1 = \sigma_1^2(1, 0) + 1 = 1 + 1 = 2$$

$$god(1, 0) = \sigma_1^2(2 - 1, \sigma_1^2(1, 0)) + 1 = \sigma_1^2(1, 1) + 1 = 4 + 1 = 5$$



Ejemplo Decodificación de Gödel

Decodificación de Gödel, degödelización (degod):

$\text{degod}: N^2 \rightarrow N$

$$\text{degod}(z, k) = \begin{cases} 0 & \text{si } z = 0 \vee k > l(z) \\ \sigma_{l(z),k}^1(\sigma_{2,2}^1(z-1)) & \text{resto} \end{cases}$$

con $l: N \rightarrow N$

$$l(0) = 0$$

$$l(z) = \sigma_{2,1}^1(z-1) + 1 \text{ (para } z > 0)$$

degod(2,1) =

$$\sigma_{l(2),1}^1(\sigma_{2,2}^1(1)) = \sigma_{2,1}^1(\sigma_{2,2}^1(1)) = \sigma_{2,1}^1(0) = 0$$

$$l(2) = \sigma_{2,1}^1(1) + 1 = 2$$

$$\sigma_{2,1}^1(1) = d(1) - \sigma_{2,2}^1(1) = 1 - 0 = 1$$

$$\sigma_{2,1}^1(1) = 1 - \sigma_1^2(d(1), 0) = 1 - \sigma_1^2(1, 0) = 1 - 1 = 0$$

$$\sigma_{2,1}^1(0) = d(0) - \sigma_{2,2}^1(0) = 0 - 0 = 0$$



Función codificación de sentencias y códigos (codi), (Codi)

Definición 19: Función codificación de sentencias (codi)

$\text{codi}: \text{Cód_While} \rightarrow N$

$$\text{codi}(s) = \begin{cases} 5(i-1) & \text{si } s = X_i := 0 \\ 5\sigma_1^2(i-1, j-1) + 1 & \text{si } s = X_i := X_j \\ 5\sigma_1^2(i-1, j-1) + 2 & \text{si } s = X_i := X_j + 1 \\ 5\sigma_1^2(i-1, j-1) + 3 & \text{si } s = X_i := X_j - 1 \\ 5\sigma_1^2(i-1, \text{Codi}(s_1)) + 4 & \text{si } s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od} \end{cases}$$

Definición 20: Función codificación de códigos (Codi)

Si s es un código While con $s = s_1 ; s_2 ; \dots ; s_m$ donde cada s_i es una sentencia, entonces:

$$\text{Codi}(s) = \mathbf{god}(\text{codi}(s_1), \text{codi}(s_2), \dots, \text{codi}(s_m)) - 1$$



Función codificación de programas (CODI)

Definición 21: Función codificación de programas (CODI)

Sea $Q \in \text{WHILE}$, con $Q = (n, p, s)$. Definimos la función codificación de programas, notada CODI, como:
CODI: $\text{WHILE} \rightarrow N$

$$\text{CODI}(Q) = \sigma_1^3 (n, p - \max\{n, k\}, \text{Codi}(s))$$

donde $k = \max\{m \in \Sigma_d^+ \mid X_m \succ s\}$



Ejemplo CODI

Calcular, mostrando los pasos intermedios, **CODI**(1, 2, $X_1 := X_1$).

$$\mathbf{CODI}(1, 2, X_1 := X_1) = \sigma_1^3(1, 2 - \max\{1, 1\}, \text{Codi}(X_1 := X_1)) = \sigma_1^3(1, 1, 2) = \sigma_1^2(1, \sigma_1^2(1, 2)) = \sigma_1^2(1, 8) = 53.$$

- $\text{Codi}(X_1 := X_1) = \text{god}(\text{codi}(X_1 := X_1)) - 1 = \text{god}(1) - 1 = 2$
- $\text{codi}(X_1 := X_1) = 5 \sigma_1^2(0, 0) + 1 = 1$
- $\text{god}(1) = \sigma_1^2(1-1, \sigma_1^1(1)) + 1 = \sigma_1^2(0, 1) + 1 = 3.$



Problema de la Parada



Definición de Problema

Dado un predicado P de n argumentos, y un vector $\underline{a} \in N^n$, el **problema** P asociado a dicho predicado, es la siguiente pregunta:

- ¿Se cumple el predicado P para el vector a ? o ¿ $P(a)$ es verdadero ?

Problema	Predicado	Función
resoluble	Decidible	$\in$ T-WHILE
parcialmente resoluble	Enumerable	$\in$ WHILE
no resoluble	no decidible	$\notin$ T-WHILE
totalmente no resoluble	no enumerable	$\notin$ WHILE

La tabla la podemos leer que es un problema es resoluble si el predicado es decidible y la función asociada a T-WHILE, en caso que la función no pertenezca a T-WHILE sería no resoluble pero puede ser parcialmente resoluble ya que la función puede pertenecer a WHILE.



Problema de la parada, Predicado H^n

Definición 18: Predicado H^n

Sea g el número de un programa WHILE de n entradas y x un vector de entrada para g ($x \in N^n$).

$H^n(g, x)$ es verdadero sii el programa g con la entrada x termina.

El problema de la parada para programas con n entradas es el problema asociado al predicado H^n .

Nota: $\text{Seq } Q \in \text{WHILE} \mid \text{CODI}(Q) = g$.



H^1 no resoluble

Teorema 1: H^1 no resoluble.

Demostración: Por reducción al absurdo.

H^1 es resoluble (Problema) $\Rightarrow H^1$ es decidable (Predicado) $\Rightarrow \exists H^1 \in \text{T-WHILE} \mid H^1$ es la función característica del predicado H^1 .

$$H^1(g, x) = \begin{cases} 1 & \text{si el programa de número } g \text{ con la entrada } x \text{ termina} \\ 0 & \text{si el programa de número } g \text{ con la entrada } x \text{ no termina} \end{cases}$$



H^1 no resoluble

Definimos la función **mala** como:

$$\text{mala}(x) = \begin{cases} 1 & \text{si } H^1(x, x) = 0 \\ \uparrow & \text{si } H^1(x, x) = 1 \end{cases}$$

La función **mala** es calcula el macroprograma **Mala** = (1,2,s) con s:

```
 $X_2 := X_2 + 1;$   
While  $H^1(X_1, X_1) \neq 0$  do  
   $X_2 := 0$   
od;  
 $X_1 := X_2$ 
```

Sea **m** el número del programa **Mala**. ¿Cuál es el valor de $H^1(\mathbf{m}, \mathbf{m})$?

- Si $H^1(\mathbf{m}, \mathbf{m}) = 1 \Rightarrow$ el programa **m** con la entrada **m** termina $\Rightarrow \text{mala}(\mathbf{m}) = \uparrow \Rightarrow H^1(\mathbf{m}, \mathbf{m}) = 0. \rightarrow \leftarrow$
- Si $H^1(\mathbf{m}, \mathbf{m}) = 0 \Rightarrow$ el programa **m** con la entrada **m** no termina $\Rightarrow \text{mala}(\mathbf{m}) = 1 \Rightarrow H^1(\mathbf{m}, \mathbf{m}) = 1. \rightarrow \leftarrow$

Nota: Este resultado es extensible a H^n .



Función castor afanoso



Longitud de un programa WHILE (long), P^n

Vamos a definir primero dos conceptos que necesitamos para definir la función castor afanoso.

Definición 19: Longitud de un programa WHILE (long), P^n

Sea $Q \in \text{WHILE}$, con $Q = (n, p, s)$.

$\text{long} : \text{WHILE} \rightarrow \mathbb{N}$

$\text{long}(Q) = |s|_{\text{do}} + |s|_{\text{:=}}$

$P^n = \{ Q \in \text{WHILE} \mid \text{long}(Q) = n \}$, es decir, al conjunto de todos los programas de longitud n .



Función castor afanoso

En este apartado vamos a ver una **función no computable**; es decir, una función que no es WHILE-computable

Definición 20: Función Castor Afanoso

Definimos la función castor afanoso, notada Σ , como: $\Sigma: N \rightarrow N$

$$\Sigma(n) = \max \{ Q(0) \mid Q \in \text{WHILE}^1 \wedge Q \in P^n \wedge Q(0) \in N \}$$

$\Sigma(n)$ es el mayor valor, de todos los programas WHILE de una entrada = 0 (WHILE^1) de longitud exactamente n que finaliza.



Función castor afanoso. $\Sigma(1)$

$\Sigma(1) = (1, p, s)$

(1,1,s)

S: $x_1 := 0 \rightarrow 0$

S: $x_1 := x_1 \rightarrow 0$

S: $x_1 := x_1 - 1 \rightarrow 0$

S: $x_1 := x_1 + 1 \rightarrow 1$

(1,2,s)

S: $x_1 := 0 \mid x_2 := 0 \rightarrow 0$

S: $x_1 := x_2 \mid x_2 := x_1 \rightarrow 0$

S: $x_1 := x_2 - 1 \mid x_2 := x_2 - 1 \mid x_1 := x_1 - 1 \mid x_2 := x_1 - 1 \rightarrow 0$

S: $x_2 := x_2 + 1 \mid x_2 := x_1 + 1 \rightarrow 0$

S: $x_1 := x_2 + 1 \mid x_1 := x_1 + 1 \rightarrow 1$



Función castor afanoso. $\Sigma(2)$

$$\Sigma(2) = (\mathbf{1}, \mathbf{1}, \mathbf{s})$$

$\mathbf{s}: x_1 := 0; x_1 := 0 \mid x_1 := 0; x_1 := x_1 \mid x_1 := 0; x_1 := x_1 - 1 \rightarrow 0$
 $x_1 := 0; x_1 := x_1 + 1 \rightarrow 1$

$\mathbf{s}: x_1 := x_1; x_1 := 0 \mid x_1 := x_1; x_1 := x_1 \mid x_1 := x_1; x_1 := x_1 - 1 \rightarrow 0$
 $x_1 := x_1; x_1 := x_1 + 1 \rightarrow 1$

$\mathbf{s}: x_1 := x_1 - 1; x_1 := 0 \mid x_1 := x_1 - 1; x_1 := x_1 \mid x_1 := x_1 - 1; x_1 := x_1 - 1 \rightarrow 0$
 $x_1 := x_1 - 1; x_1 := x_1 + 1 \rightarrow 1$

$\mathbf{s}: x_1 := x_1 + 1; x_1 := 0 \mid x_1 := x_1 + 1; x_1 := x_1 - 1 \rightarrow 1$
 $x_1 := x_1 + 1; x_1 := x_1 - 1 \rightarrow 0$
 $x_1 := x_1 + 1; x_1 := x_1 + 1 \rightarrow \mathbf{2}$
...



Función castor afanoso. $\Sigma(2)$

$\Sigma(2) = (1, 1, s)$

...

s: while $x_1 \neq 0$ do $x_1 := 0$; od $\rightarrow 0$

s: while $x_1 \neq 0$ do $x_1 := x_1$; od $\rightarrow 0$

s: while $x_1 \neq 0$ do $x_1 := x_1 - 1$; od $\rightarrow 0$

s: while $x_1 \neq 0$ do $x_1 := x_1 + 1$; od $\rightarrow 0$

Hasta aquí sin problemas :-).



Función castor afanoso. $\Sigma(3)$

$\Sigma(3) = (1, 1, s)$

Asignaciones: Máximo $X_1 := X_1 + 1; X_1 := X_1 + 1; X_1 := X_1 + 1 \rightarrow 3$

...

While:

s: $x_1 := 0$; while $x_1 \neq 0$ do *<cualquier asignación>* od $\rightarrow 0$

s: $x_1 := x_1$; while $x_1 \neq 0$ do *<cualquier asignación>* od $\rightarrow 0$

s: $x_1 := x_1 - 1$; while $x_1 \neq 0$ do *<cualquier asignación>* od $\rightarrow 0$

s: $x_1 := x_1 + 1$; while $x_1 \neq 0$ do $x_1 := 0$ od $\rightarrow 0$

s: $x_1 := x_1 + 1$; while $x_1 \neq 0$ do $x_1 := x_1 - 1$ od $\rightarrow 0$

s: $x_1 := x_1 + 1$; while $x_1 \neq 0$ do $x_1 := x_1$ od $\rightarrow \uparrow$

s: $x_1 := x_1 + 1$; while $x_1 \neq 0$ do $x_1 := x_1 + 1$ od $\rightarrow \uparrow$

...



Función castor afanoso

Proposición 1: $\Sigma(n+1) > \Sigma(n) \forall n \in \mathbb{N}$. La función castor afanoso es estrictamente creciente.

Demostración: Sea $Q = (1, p, s)$ un programa con $\text{long}(Q) = n$ y que $Q(0) = \Sigma(n)$.

Sea $Q' = (1, p, s')$ un programa, con s' :

s ;
 $X_1 := X_1 + 1$

$$\begin{aligned} \text{long}(Q') &= n+1 \wedge Q'(0) = \Sigma(n) + 1 \Rightarrow \\ \Sigma(n+1) &\geq \Sigma(n) + 1 > \Sigma(n) \Rightarrow \\ \Sigma(n+1) &> \Sigma(n) \end{aligned}$$

- La salida de Q' será el castor afanoso +1 ya que $Q(0) = \Sigma(n)$.
- Hemos demostrado que Castor afanoso de $(n) + 1$ es mayor que el castor afanoso de (n) . Esa será la cota inferior del castor afanoso de $(n+1)$, ya que puede ser mayor que castor afanoso $(n)+1$.
- Por tanto, castor afanoso $(n+1)$ es mayor que castor afanoso (n) .



Función castor afanoso

Proposición 2: $\exists Q \in P^k \mid f = F_Q \Rightarrow \Sigma(n+k) \geq f(n) \forall n \in N$

Demostración: Sea $Q = (1, p, s)$ un programa con $\text{long}(Q) = k$ y $F_Q = f$.

Sea $Q' = (1, p, s')$ un programa, con s' :

$$\left. \begin{array}{l} X_1 := X_1 + 1 \\ \dots \\ X_1 := X_1 + 1 \end{array} \right\} \begin{array}{l} n \text{ veces} \\ s; \end{array}$$

- Asignamos n a X_1 con lo cuál s comienza con $x_1 = n$, que sería equivalente a calcular $f(n)$ ya que s calcula f .
- Establecemos una cota superior para la función castor afanoso de $(n+k)$, al menos sería $f(n)$.

Se cumple que: $\text{long}(Q') = n+k \wedge Q'(0) = f(n) \Rightarrow \Sigma(n+k) \geq f(n)$



Función castor afanoso. $\Sigma \notin F(WHILE)$

Teorema 2: $\Sigma \notin F(WHILE)$ No computable

Demostración: Lo demostraremos por reducción al absurdo. **Supongamos que:** $\Sigma \in F(WHILE)$

Definimos dos funciones:

- Sea $g(n)=2n$; $g \in F(WHILE)$. La función doble es computable.
- Sea $f(n)=\Sigma(2n)$; $f \in F(WHILE)$. Ya que es la composición de dos funciones computable y es una función computable.

$$\exists Q \in P^k \mid f = F_Q \Rightarrow \Sigma(n+k) \geq f(n) \forall n \in N \Rightarrow \Sigma(n+k) \geq \Sigma(2n) \forall n \in N \Rightarrow \Sigma(2k+1) \geq \Sigma(2k+2)$$

Proposición 2

$$f(n) = \Sigma(2n)$$

$$n = k + 1$$

$\Sigma(2k+1) \geq \Sigma(2k+2) \not\equiv \Sigma(2k+1) < \Sigma(2k+2)$. Por proposición 1. Por tanto, $\Sigma \notin F(WHILE)$



$\Sigma \notin F(WHILE) \Rightarrow H^1$ no es resoluble.

Proposición 3: $\Sigma \notin F(WHILE) \Rightarrow H^1$ no es resoluble.

Demostración: Lo demostraremos por reducción al absurdo. **Supongamos que:** $\Sigma \notin F(WHILE)$, **que** H^1 **es resoluble.**

En ese caso puedo seleccionar, para cada longitud dada, sólo los programas que terminan para el cero como entrada. Tras ejecutarlos para el cero, escojo la mayor salida.

Esto implica que: $\Sigma \in F(WHILE) \rightarrow \leftarrow$. Por tanto:

$\Sigma \notin F(WHILE) \Rightarrow H^1$ no es resoluble.



Complejidad Temporal





Complejidad

- La complejidad computacional es un campo de la informática que analiza algoritmos en función de la cantidad de recursos necesarios para ejecutarlos. La cantidad de recursos necesarios varía según el tamaño de la entrada, por lo que la complejidad generalmente se expresa como una función de **n** , donde **n** es el tamaño de la entrada.
- Es importante tener en cuenta que al analizar un algoritmo podemos considerar **la complejidad temporal y la complejidad espacial**.
- La **complejidad espacial** es básicamente la cantidad de espacio de memoria necesario para resolver un problema en relación con el tamaño de la entrada.
- Aunque la **complejidad espacial** es importante al analizar un algoritmo, en esta historia nos enfocaremos únicamente en la **complejidad temporal**.



Complejidad Temporal

- La complejidad temporal mide cómo aumenta el tiempo de ejecución de un algoritmo en relación con el tamaño de la entrada. Nos ayuda a comprender qué tan eficiente es un algoritmo y cómo se escala a medida que aumenta el tamaño de la entrada.
- Cuando analizamos la complejidad temporal, **consideramos el peor caso**, ya que proporciona un límite superior para el tiempo de ejecución de cualquier entrada de un tamaño determinado. Este análisis del peor caso garantiza que el algoritmo siempre finalizará en un tiempo determinado, independientemente de la entrada específica.
- La complejidad temporal generalmente se expresa utilizando la notación Big O, que describe el límite superior de la tasa de crecimiento de un algoritmo. Por ejemplo, si un algoritmo tiene una complejidad temporal de $O(n)$, significa que el peor tiempo de ejecución aumenta linealmente con el tamaño de la entrada.



Complejidad Temporal

Notación Big O	Nombre	Ejemplos
$O(1)$	Constante	Acceso a un elemento de un array
$O(\log n)$	Logarítmico	Búsqueda Binaria
$O(n)$	Lineal	Búsqueda Lineal
$O(n \log n)$	Quasilineal	Mergesort
$O(n^2)$	Cuadrático	Ordenamiento burbuja
$O(2^n)$	Exponencial	Fibonacci
$O(n!)$	Factorial	<i>Buscar ejemplo</i>

La siguiente página muestra la complejidad temporal (Big O) de varios operadores nativos en python: <https://wiki.python.org/moin/TimeComplexity>



Complejidad Temporal. $O(1)$ - $O(\log n)$

- $O(1)$ - **Tiempo constante**: Los algoritmos con complejidad temporal constante tienen un tiempo de ejecución fijo, independientemente del tamaño de la entrada.

```
def get_first(data):  
    return data[0]  
data = [10, 7, 91, 5, 13, 44, 37, 86, 55]  
print(get_first(data))
```

- $O(\log n)$ - **Tiempo logarítmico**: Los algoritmos con complejidad temporal logarítmica a menudo dividen la entrada en partes más pequeñas, reduciendo el espacio de búsqueda en cada paso. A medida que el tamaño de la entrada aumenta, el tiempo de ejecución crece, pero a una velocidad mucho más lenta que el tiempo lineal.

```
data = [10, 7, 91, 5, 13, 44, 37, 86, 55]  
for index in range(0, len(data), 3):  
    print(data[index])
```



Complejidad Temporal. $O(n)$ - $O(n \log n)$

- $O(n)$ - **Tiempo lineal**: Los algoritmos con complejidad temporal lineal tienen un tiempo de ejecución proporcional al tamaño de la entrada. A medida que la entrada crece, el tiempo de ejecución aumenta al mismo ritmo.

```
data = [10, 7, 91, 5, 13, 44, 37, 86, 55]
for value in data:
    print(value)
```

- $O(n \log n)$ - **Tiempo quasilineal**: Los algoritmos con complejidad temporal quasilineal tienen un tiempo de ejecución que crece de forma casi lineal con el tamaño de la entrada, pero también depende del logaritmo de la entrada. Son comunes en algoritmos de ordenación eficientes, como el algoritmo de ordenación rápida (quicksort) y el algoritmo de fusión (merge sort).

```
data = [10, 7, 91, 5, 13, 44, 37, 86, 55]
datab = [10, 7, 91, 5, 13, 44, 37, 86, 55]
for value in data: # $O(n)$ 
    result.append(busqueda_binaria(datab, value)) # $O(\log n)$ 
```



Complejidad Temporal. $O(n^2)$

- $O(n^2)$ - **Tiempo cuadrático**: Los algoritmos con complejidad temporal cuadrática tienen un tiempo de ejecución que aumenta proporcionalmente al cuadrado del tamaño de la entrada. En general, los algoritmos con tiempo cuadrático son menos eficientes y pueden volverse impracticables para tamaños de entrada grandes.

```
data = [10, 7, 91, 5, 13, 44, 37, 86, 55]
for x in data:
    for y in data:
        print(x, y)
```



Complejidad Temporal. $O(2^n)$ - $O(n!)$

- $O(2^n)$ - **Tiempo exponencial**: Los algoritmos con complejidad temporal exponencial tienen un tiempo de ejecución que crece exponencialmente con el tamaño de la entrada. Estos algoritmos pueden ser altamente ineficientes incluso para tamaños de entrada moderados, lo que los hace no adecuados para muchos problemas del mundo real.

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    return fibonacci(n-1) + fibonacci(n-2)
```

- $O(n!)$ - **Tiempo factorial**: Los algoritmos con complejidad temporal factorial tienen un tiempo de ejecución que crece factorialmente con el tamaño de la entrada. Estos algoritmos son extremadamente ineficientes y a menudo no son factibles incluso para entradas relativamente pequeñas.



Calculabilidad





Prueba por contradicción

Una prueba por contradicción siempre prueba que alguna sentencia dada es falsa. Para probar que la sentencia es verdadera, debo primero asumir lo contrario.

- S = sentencia que queremos probar que es falsa.
p.ej.: "Fui a ver al Unicaja el Sábado al Martín Carpena"
- C = consecuencia que sigue cuando S es verdadera
p.ej.: "Estaba en Málaga el Sábado."
- T = un hecho que se conoce que es verdad, pero contradice C
p.ej.: "Estaba jugando el Open de Australia el Sábado"
- Razonamiento: Si S fuera verdadero, entonces C sería verdadero, pero es imposible ya que T es verdadero y contradice C , por lo que S debe ser falso.
*p.ej. "Si **fuí a ver al Unicaja el Sábado al Martín Carpena**, entonces estaba en Málaga el Sábado, pero estaba jugando el Open de Australia el Sábado, por lo que **no fui a ver al Unicaja el Sábado al Martín Carpena.**"*



Programas python auxiliares

Vamos a ver una serie de programas en Python:

- Un programa que cuenta las líneas de un fichero.

```
def countLines(inString):  
    lines = inString.split('\n')  
    return str(len(lines))
```

- Un programa que lee un fichero.

```
def readFile(fileName):  
    fileContents = ''  
    with open(fileName) as inputFile:  
        fileContents = inputFile.read()  
    return fileContents
```



Programas python auxiliares

- Un programa que multiplica una cadena de números: » `multiplicaTodo("2 3 2 10")` » "120".

```
def multiplyAll(inString):  
    numbers = inString.split()  
    for i in range(len(numbers)):  
        numbers[i] = int(numbers[i])  
    product = 1  
    for num in numbers:  
        product = product * num  
        productString = str(product)  
    return productString
```

- Un programa que analiza a otro programa:
» `countLines(readFile("multiplyAll.py"))`
- Un programa que se analiza a el mismo:
» `countLines(readFile("countLines.py"))`



Programas de decisión

Vamos a ver cuatro programas de decisión que necesitaremos. Al ser programas de decisión su salida será yes/no.

```
def containsGAGA(inString):  
    if 'GAGA' in inString:  
        return 'yes'  
    else:  
        return 'no'
```

```
def yes(inString):  
    return 'yes'
```

```
def longerThan1K(inString):  
    if len(inString) > 1000:  
        return 'yes'  
    else:  
        return 'no'
```



Programas de decisión

```
def maybeLoop(inString):  
    if not 'secret sauce' in inString:  
        # enter an infinite loop  
        i = 0  
        while i>=0:  
            i = i + 1  
    else:  
        # output ``yes'' if input length is even and ``no'' otherwise  
        if len(inString) % 2 == 0:  
            return 'yes'  
        else:  
            return 'no'
```



Resultado de Programas analizando otros programas

Ejecución (rf = readFile)	Salida
containsGAGA('CTGAGAT')	yes
containsGAGA(rf('geneticString.txt'))	yes
containsGAGA(rf('longerThan1K.py'))	no
containsGAGA(rf('containsGAGA.py'))	yes
yes('CTGAGAT')	yes
yes(rf('geneticString.txt'))	yes
yes(rf('containsGAGA.py'))	yes
yes(rf('yes.py'))	yes
longerThan1K('CTGAGAT')	no
longerThan1K(rf('geneticString.txt'))	yes
longerThan1K(rf('containsGAGA.py'))	no
longerThan1K(rf('longerThan1K.py'))	no
maybeLoop('CTGAGAT')	undefined
maybeLoop('some secret sauce')	no
maybeLoop(rf('containsGAGA.py'))	undefined
maybeLoop(rf('maybeLoop.py'))	yes

Las ejecuciones definidas como **undefined** se deben a que el programa no termina y se queda en un bucle infinito.



yesOnString (P,I)

Programa recibe dos cadenas como parámetros, P (Programa) e I.

$$yesOnString.py(P, I) = \begin{cases} \text{"yes"} & \text{Si P es un programa Python, P(I) está definido} \\ & \text{y P(I) = "yes"} \\ \text{"no"} & \text{en otro caso.} \end{cases} \quad (1)$$

Ejemplo: `yesOnString(rf('containsGAGA.py'),'CAGT')`.

- $P = \text{containsGAGA}$ e $I = \text{'CAGT'}$. $P(I) = \text{containsGAGA('CAGT')} = \text{no}$
- $\text{yesOnString}(\text{rf('containsGAGA.py')}, \text{'CAGT'}) = \text{no}$



yesOnString (P,I)

Programa recibe dos cadenas como parámetros, P (Programa) e I.

$$\text{yesOnString.py}(P, I) = \begin{cases} \text{"yes"} & \text{Si P es un programa Python, P(I) está definido} \\ & \text{y P(I) = "yes"} \\ \text{"no"} & \text{en otro caso.} \end{cases} \quad (2)$$

Ejecución (rf = readFile)	P(I)	Salida
yesOnString('not a program','CAGT')	-	no
yesOnString(rf('containsGAGA.py'),'CAGT')	no	no
yesOnString(rf('containsGAGA.py'),rf('containsGAGA.py'))	yes	yes
yesOnString(rf('yes.py'),'CAGT')	yes	yes
yesOnString(rf('yes.py'),rf('yes.py'))	yes	yes
yesOnString(rf('longerThan1K.py'),rf('geneticString.txt'))	yes	yes
yesOnString(rf('longerThan1K.py'),rf('longerThan1K.py'))	no	no



yesOnSelf

Programa recibe una cadena como parámetros, P (Programa).

$$yesOnSelf.py(P) = \begin{cases} \text{"yes"} & \text{Si P es un programa Python, P(P) está definido} \\ & \text{y P(P) = "yes"} \\ \text{"no"} & \text{en otro caso.} \end{cases} \quad (3)$$

Ejecución (rf = readFile)	P(P)	Salida
yesOnSelf('not a program')	-	no
yesOnSelf(rf('containsGAGA.py'))	yes	yes
yesOnSelf(rf('yes.py'))	yes	yes
yesOnSelf(rf('longerThan1K.py'))	no	no
yesOnSelf(rf('yesOnSelf.py'))	yes o no?	yes o no?

Veremos que una correcta versión de *yesOnSelf.py* no es posible.



notYesOnSelf

Programa recibe una cadena como parámetros, P (Programa).

$$\text{notYesOnSelf.py}(P) = \begin{cases} \text{"no"} & \text{Si P es un progrma Python, P(P) está definido} \\ & \text{y P(P) = "yes"} \\ \text{"yes"} & \text{en otro caso.} \end{cases} \quad (4)$$

Ejecución (rf = readFile)	P(P)	Salida
notYesOnSelf('not a program')	-	yes
notYesOnSelf(rf('containsGAGA.py'))	yes	no
notYesOnSelf(rf('yes.py'))	yes	no
notYesOnSelf(rf('longerThan1K.py'))	no	yes
notYesOnSelf(rf('notYesOnSelf.py'))	yes o no?	!?!?!?!?

Al ser $P = \text{notYesOnSelf.py}$ tenemos una contradicción, con lo cual *notYesOnSelf.py* **no puede existir**.



notYesOnSelf

Al ser $P = \text{notYesOnSelf.py}$ tenemos una contradicción, con lo cual *notYesOnSelf.py* **no puede existir**.

$$\text{notYesOnSelf}(\text{notYesOnSelf}) = \begin{cases} \text{"no"} & \text{notYesOnSelf}(\text{notYesOnSelf}) = \text{"yes"} \\ \text{"yes"} & \text{en otro caso.} \end{cases} \quad (5)$$



Y si `yesOnstring` no existe?

Si asumimos que `yesOnString.py` existe, entonces podemos definir `yesOnSelf` y `notYesOnSelf` con las siguientes funciones:

```
def yesOnSelf(progString):  
    return yesOnString(progString, progString)
```

```
def notYesOnSelf(progString):  
    val = yesOnSelf(progString)  
    if val == 'yes':  
        return 'no'  
    else:  
        return 'yes'
```

Pero esto contradice nuestro resultado previo donde vimos que `notYesOnSelf.py` no puede existir. Por tanto, nuestra suposición inicial era incorrecta, por lo que concluimos que `yesOnString.py` no puede existir tampoco.



Vamos a definir weirdYesOnString

Vamos a llegar a la misma conclusión que en la transparencia anterior, con una versión más compacta. Para ello definimos *weirdYesOnString*, asumiendo que existe *yesOnString.py*

```
def weirdYesOnString(progString):  
    if yesOnString(progString, progString)=='yes':  
        return 'no'  
    else:  
        return 'yes'
```

Este programa tiene el mismo comportamiento que *notYesOnSelf.py*: cuándo ejecutamos *weirdYesOnString(weirdYesOnString)*, devuelve "si" si el mismo devuelve "no". Por tanto, todos los posibles comportamientos resultan en una contradicción.

$$\text{weirdYesOnString}(\text{weirdYesOnString}) = \begin{cases} \text{"no"} & \text{weirdYesOnString}(\text{weirdYesOnString}) = \text{"yes"} \\ \text{"si"} & \text{en otro caso.} \end{cases} \quad (6)$$



On computable numbers

La primera persona en publicar una prueba de este estilo fue Alan Turing, en un artículo de 1936 "On computable numbers", e incluye la siguiente reflexión que podemos adoptar nosotros:

This proof, although perfectly sound, has the disadvantage that it may leave the reader with a feeling that "there must be something wrong".

La idea de incluir la demostración "larga" es la introducir los siguientes conceptos:

- *yesOnString.py* presenta la idea los programas pueden analizar otros programas y mostrar que salida daría para varias entradas;
- *yesOnSelf.py* presenta la idea que podemos analizar como un programa se comporta cuando le damos el mismo programa como entrada. Este idea es conocida como "diagonalización", en referencia a la demostración matemática inventada por el matemático Georg Cantor;
- *notYesOnSelf.py* presenta la idea de invertir la salida de un programa de decisión, específicamente para producir un programa que se auto contradiga.



Los programas perfectos para encontrar errores son imposibles

Basándonos en las definiciones previas, vamos a definir *crashOnString.py*, *crashOnSelf.py* y *weirdCrashOnSelf.py* y probaremos que *crashOnString.py* no puede existir.

$$\text{crashOnString}(P, I) = \begin{cases} \text{"yes"} & P(I) \text{ lanza una excepción en python} \\ \text{"no"} & \text{en otro caso.} \end{cases} \quad (7)$$

$$\text{crashOnSelf}(P) = \begin{cases} \text{"yes"} & P(P) \text{ lanza una excepción en python} \\ \text{"no"} & \text{en otro caso.} \end{cases} \quad (8)$$

$$\text{weirdCrashOnSelf}(P) = \begin{cases} \text{"sin excepción"} & \text{Si } P(P) \text{ lanza una excepción en python} \\ \text{"excepción"} & \text{en otro caso.} \end{cases} \quad (9)$$



crashOnSelf y weirdCrashOnSelf

La definición en python de *crashOnSelf.py* y *weirdCrashOnSelf.py* son:

```
def crashOnSelf(progString):  
    return crashOnString(progString, progString)  
  
def weirdCrashOnSelf(progString):  
    val = crashOnSelf(progString)  
    if (val == 'yes'):  
        return 'did not crash!'  
    else:  
        x = 1 / 0 # deliberately crash (divide by zero)
```



weirdCrashOnSelf

Prueba de la afirmación. Esta prueba es por contradicción. Asumimos que weirdCrash-OnSelf.py existe.

```
>>> weirdCrashOnSelf(rf('weirdCrashOnSelf.py')) [W]
```

1. W lanza excepción. Si W lanza una excepción, val = crashOnSelf(progString) devuelve yes y el programa termina sin lanzar una excepción (devuelve "did not crash!"). En resumen, si W lanza una excepción, entonces no lanza la excepción, por lo que llegamos a una contradicción.
2. W no lanza excepción. Si W no lanza una excepción, val = crashOnSelf(progString) devuelve "no", pero el programa realiza una división por cero. En resumen, si W no lanza una excepción, entonces manda una excepción.

```
val = crashOnSelf(progString)
```

Las dos posibilidades terminan en contradicciones, por lo que concluimos que *weirdCrashOnSelf.py* no puede existir.



crashOnString

El programa *crashOnString.py* no puede existir.

Prueba de la afirmación. Esta es una prueba por contradicción. Asumimos que *crashOnString.py* no existe.

Entonces fácilmente podemos escribir los programas *crashOnSelf.py* y *weirdCrashOnSelf.py* mostradas en las transparencias anteriores.

Esto contradice que *weirdCrashOnSelf.py* no puede existir.

Por tanto, nuestra suposición inicial era incorrecta, y concluimos que *crashOn-String.py* no puede existir tampoco.



That's all folks!

